

GitHub Repo: <https://github.com/matthewwoo527/cs240-assignment3>

Matthew Woo-Assignment 3

CS 240

1. Install or open MARS and run a Hello World program.

```
1  .data
2      helloworld: .ascii "Hello world"
3
4  .text
5  li $v0, 4
6  la $a0, helloworld
7
8  syscall
9
```

Output:

Mars Messages Run I/O

```
Hello world
-- program is finished running (dropped off bottom) --
```

Name	Number	Value
\$zero	0	0
\$at	1	268500992
\$v0	2	4
\$v1	3	0
\$a0	4	268500992
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	0
\$t1	9	0
\$t2	10	0
\$t3	11	0
\$t4	12	0
\$t5	13	0
\$t6	14	0
\$t7	15	0
\$s0	16	0
\$s1	17	0
\$s2	18	0
\$s3	19	0
\$s4	20	0
\$s5	21	0
\$s6	22	0
\$s7	23	0
\$t8	24	0
\$t9	25	0
\$k0	26	0
\$k1	27	0
\$gp	28	268468224
\$sp	29	2147479548
\$fp	30	0
\$ra	31	0
pc		4194320
hi		0
lo		0

Reflection:

We can see that register \$v0 set to 4, and \$a0 stores address of null-terminated string to print. We set \$a0 to the address represented by the label called helloworld (.asciiz “Hello world” will give us an address that stores ascii value).

2. Write a loop that prints the integers 1 through 100.

```

1  li $t0, 1
2  li $t1, 101
3
4  loop:
5  j PrintItem
6  PrintItemEnd:
7  #increment
8  addi $t0, $t0, 1
9  bne $t0, $t1, loop
10
11 j Exit
12
13 PrintItem:
14 j PrintInt
15 PrintIntEnd:
16 #print new line
17 j PrintNewLine
18 PrintNewLineEnd:
19 j PrintItemEnd
20
21 PrintInt:
22 #print integer
23 li $v0, 1
24 move $a0, $t0
25 syscall
26
27 PrintNewLine:
28 li $v0, 11
29 li $a0, '\n'
30 syscall
31 j PrintNewLineEnd
32
33 Exit:
34

```

Output:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92
93
94
95
96
97
98
99
100

-- program is finished running (dropped off bottom) --

Registers Coproc 1 Coproc 0		
Name	Number	Value
\$zero	0	0
\$at	1	0
\$v0	2	11
\$v1	3	0
\$a0	4	10
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	101
\$t1	9	101

Reflection:

This program number 1 to 100. First, set \$t0 to 1 and \$t1 to 101, then PrintInt (the number in \$t0) and PrintNewLine. Add 1 to \$t0 and repeat print for each loop until \$t0 == \$t1 (when 1 = 101).

3. Write a program that calculates the sum of the even integers from 1 through 100.

```

1 #Question 3-Even add
2 li $t0, 0
3 li $t1, 102
4
5 loop:
6 add $t2, $t2, $t0
7 addi $t0, $t0, 2
8 bne $t0, $t1, loop
9 |

```

Output:

Registers Coproc 1 Coproc 0		
Name	Number	Value
\$zero	0	0
\$at	1	0
\$v0	2	0
\$v1	3	0
\$a0	4	0
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	102
\$t1	9	102
\$t2	10	2550

Reflection:

The output is at the register \$t2. Set register \$t0 to 0 and \$t1 to 102. Start from 0, add 2 in each run, $t2 = t2 + t0$. Then $t0 = t0 + 2$. If $t0 \neq t1$, then keep looping. Register \$t2 is the output/result.

4. Write a program that loads two integers from Memory and puts them into registers, adds the values together, and stores back to memory.

```

1  # [Setup]
2  li $t0, 5 #Value 1
3  li $t1, 6 #Value 2
4  li $t3, 268500992 #Address 1
5  li $t4, 268501024 #Address 2
6  sw $t0, 0($t3) #Store the value in $t0 into the address in $t3
7  sw $t1, 0($t4) #Store the value in $t0 into the address in $t4
8  # Clear so we can make sure the value is from the memory
9  li $t0, 0
10 li $t1, 0
11 li $t3, 0
12 li $t4, 0
15 # [Start program]
16 # Purpose: Pull the two summands from memory, and produce a sum
17 # and save the sum to memory
18
19 # 0. Load the two imm memory addresses into registers
20 li $t3, 268500992
21 li $t4, 268501024
22 li $t5, 268501056
23
24 # 1. Load the two summands from mem and place them into registers
25 lw $t0, 0($t3)
26 lw $t1, 0($t4)
27
28 # 2. Sum the registers together and place the sum in another register
29 add $t2, $t0, $t1 #$t2 is the destination
30
31 # 3. Store the sum into memory
32 sw $t2, 0($t5)
33

```

Registers			Coproc 1	Coproc 0
Name	Number	Value		
\$zero	0	0		
\$at	1	268500992		
\$v0	2	0		
\$v1	3	0		
\$a0	4	0		
\$a1	5	0		
\$a2	6	0		
\$a3	7	0		
\$t0	8	5		
\$t1	9	6		
\$t2	10	11		
\$t3	11	268500992		
\$t4	12	268501024		
\$t5	13	268501056		
\$t6	14	0		
\$t7	15	0		
\$s0	16	0		
\$s1	17	0		
\$s2	18	0		
\$s3	19	0		
\$s4	20	0		
\$s5	21	0		
\$s6	22	0		
\$s7	23	0		
\$t8	24	0		
\$t9	25	0		
\$k0	26	0		
\$k1	27	0		
\$gp	28	268468224		
\$sp	29	2147479548		
\$fp	30	0		
\$ra	31	0		
pc		4194392		
hi		0		
lo		0		

Output:

Output is the value in \$t2, 11

Reflection:

This program takes two integer and stores it into the memory. Then load the values from memory into register and sum that. Then store that value to memory.

5. Write a program that prints FizzBuzz.

```
1  li $t0, 1
2  li $t1, 101
3
4  loop:
5      j PrintItem
6  PrintItemEnd:
7      #increment
8      addi $t0, $t0, 1
9      bne $t0, $t1, loop
10
11     j Exit
12
13 PrintItem:
14     rem $t2, $t0, 3 #Save remainder to $t2, $t2 == $t0 & 3
15     rem $t3, $t0, 5 #Save remainder to $t3, $t3 == $t0 & 5
16     #Check 3
17     beq $t2, $zero, CheckFive #if 3 pass(can divide by 3), then check if 5 pass(can divide by 5)
18     #Check 5
19     beq $t3, $zero, PrintBuzz
20     #If not dividable by 3 or 5, then PrintInt
21     j PrintInt
22 PrintIntEnd:
23     #Check 5 if Check 3 Pass
24     CheckFive:
25         beq $t3, $zero, PrintFizzBuzz #If yes then PrintFizzBuzz
26         j PrintFizz #If not then PrintFizz because it checked 3 and passed
27     #print new line
28     # j PrintNewLine
29 PrintNewLineEnd:
30     j PrintItemEnd
31
32 PrintInt:
33     #print integer
34     li $v0, 1
35     move $a0, $t0
36     syscall
37
38 PrintNewLine:
39     li $v0, 11
40     li $a0, '\n'
41     syscall
42     j PrintNewLineEnd
43
44 PrintFizz:
45     .data
46     Fizz: .asciiz "Fizz"
47     .text
48     li $v0, 4
49     la $a0, Fizz
50     syscall
51     j PrintFizzEnd
52 PrintFizzEnd:
53     j PrintNewLine
54
55 PrintBuzz:
56     .data
57     Buzz: .asciiz "Buzz"
58     .text
59     li $v0, 4
60     la $a0, Buzz
61     syscall
62     j PrintBuzzEnd
63 PrintBuzzEnd:
64     j PrintNewLine
65
66 PrintFizzBuzz:
67     .data
68     FizzBuzz: .asciiz "FizzBuzz"
69     .text
70     li $v0, 4
71     la $a0, FizzBuzz
72     syscall
73     j PrintFizzBuzzEnd
74 PrintFizzBuzzEnd:
75     j PrintNewLine
76
77 Exit:
78
```

Registers Coproc 1 Coproc 0		
Name	Number	Value
\$zero	0	0
\$at	1	268500992
\$v0	2	11
\$v1	3	0
\$a0	4	10
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	101
\$t1	9	101
\$t2	10	1

Output:

1

2

Fizz

4

Buzz

Fizz

7

8

Fizz

Buzz

11

Fizz

13

14

FizzBuzz

16

17

Fizz

19

Buzz

Fizz

22

23

Fizz

Buzz

26

Fizz

28

29

FizzBuzz

31

32

Fizz

34

Buzz

Fizz

37

38

Fizz

Buzz

41

Fizz

43

44

FizzBuzz

46

47

Fizz

49

Buzz

Fizz

52

53

Fizz

Buzz

56

Fizz

58

59

FizzBuzz

61

62

Fizz

64

Buzz

Fizz

67

68

Fizz

Buzz

71

Fizz

73

74

FizzBuzz

76

77

Fizz

79

Buzz

Fizz

82

83

Fizz

Buzz

86

Fizz

88

89

FizzBuzz

91

92

Fizz

94

Buzz

Fizz

97

98

Fizz

Buzz

-- program is finished running (dropped off bottom) --

Reflection:

This program print Fizz when dividable by 3, print Buzz when dividable by 5, print FizzBuzz if dividable by both. First set \$t0 to 1, \$t1 to 101. Then jump PrintItem, check FizzBuzz and print Fizz or Buzz or FizzBuzz. After each runs increment by 1 and keep looping until \$t0 == \$t1 (when 1 increment to 101). We check the remainder to check if it can divided by 3 or 5. \$t2 stores the remainder for %3 for each runs and \$t3 stores the remainder for %5 for each runs.